IMEN574 / IDSC574 · PROGRAMMING FOR AX

14

Deep Learning

Sangwoo Mo · POSTECH
Spring 2027

Where We Are

01 **Programming with AI**

Lectures 2–4

02 **Python and Classical Machine Learning**

Lectures 5–13

03 **Deep Learning and Foundation Models**

Lectures 14–22

Outline: Deep Learning

01 **Learned Features**

Build a nonlinear representation

02 **Differentiable Learning**

Trace the training computation

03 **Architectural Structure**

Match computation to data

01 · Learned Features

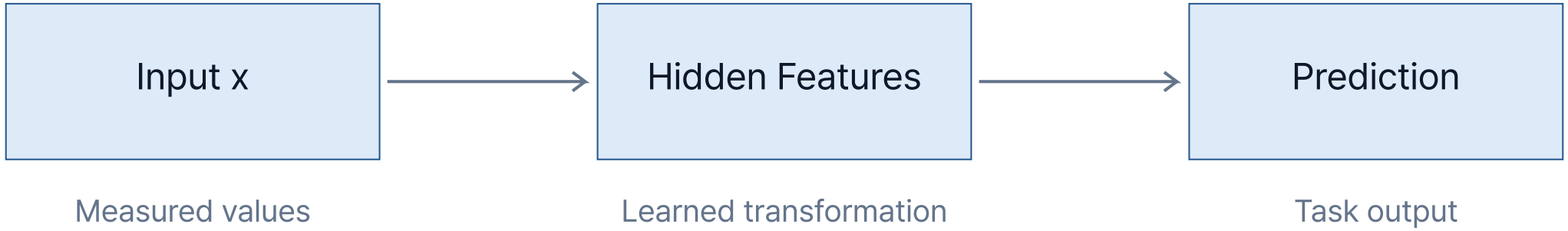
01 **Learned Features**
Build a nonlinear representation

02 **Differentiable Learning**
Trace the training computation

03 **Architectural Structure**
Match computation to data

Learning a Representation

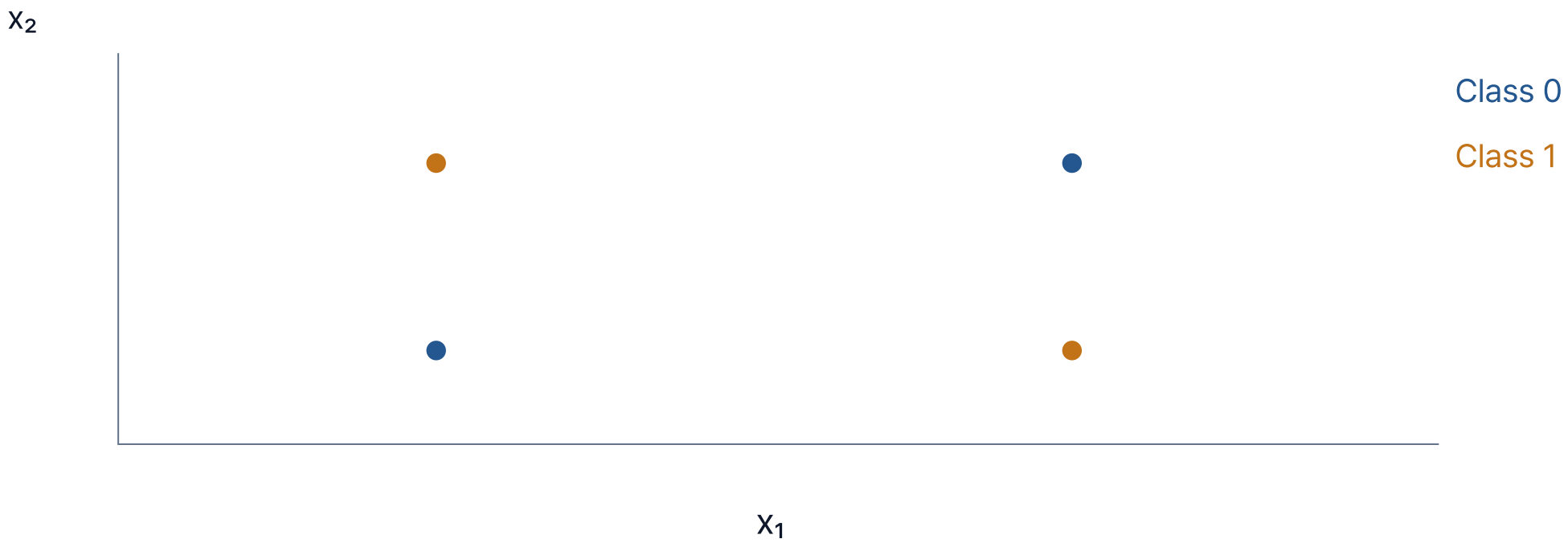
A network learns features together with a predictor



The hidden representation changes during training

A Linear Limitation

One line cannot separate the XOR pattern



Opposite corners share labels that one line cannot separate

Affine Layers

A matrix maps each input into several learned scores

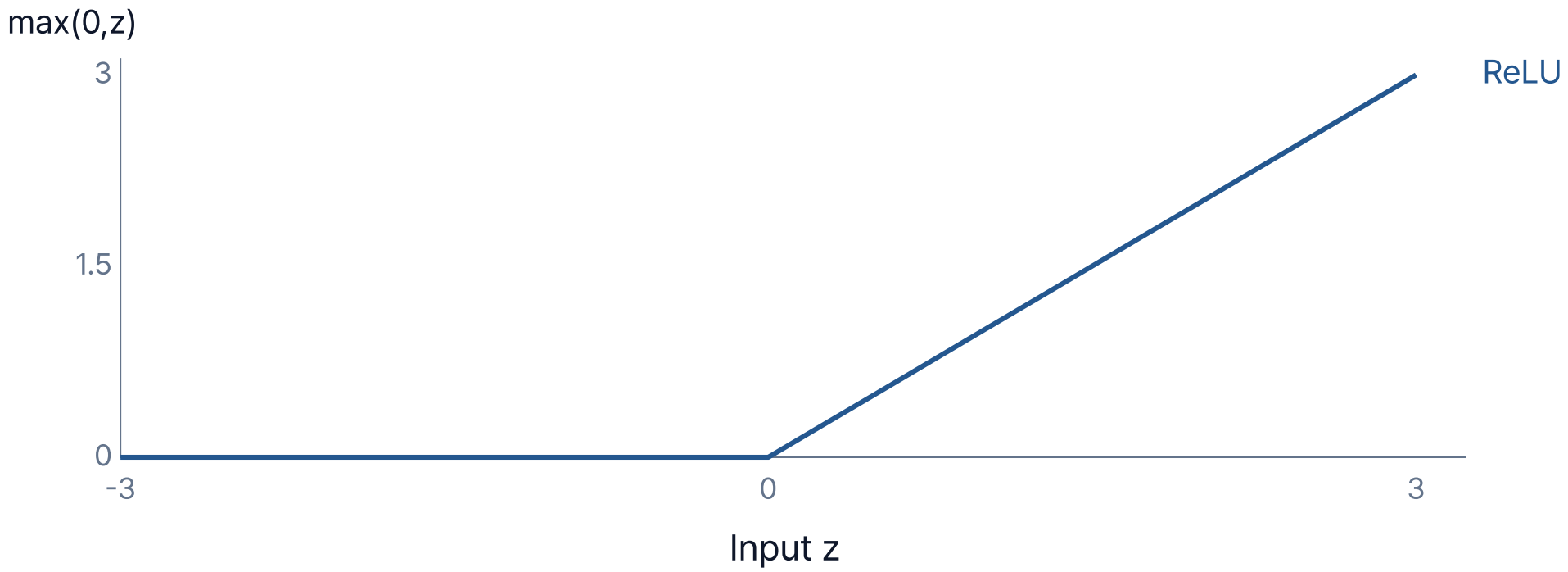
	x_1	x_2
Hidden 1	1	1
Hidden 2	1	1

$$z = Wx + b$$

Each row of W defines one feature calculation

Rectified Linear Units

Keep positive scores and remove negative scores



The kink introduces a nonlinear operation

Nonlinear Feature Construction

Two rectified features can represent the XOR target

	h_1	h_2	$h_1 - 2h_2$
(0,0)	0	0	0
(0,1)	1	0	1
(1,0)	1	0	1
(1,1)	2	1	0

$$h_1 = \mathrm{ReLU}(x_1+x_2),\ h_2 = \mathrm{ReLU}(x_1+x_2-1)$$

The transformed output matches all four XOR labels

Verifying a Tiny Network

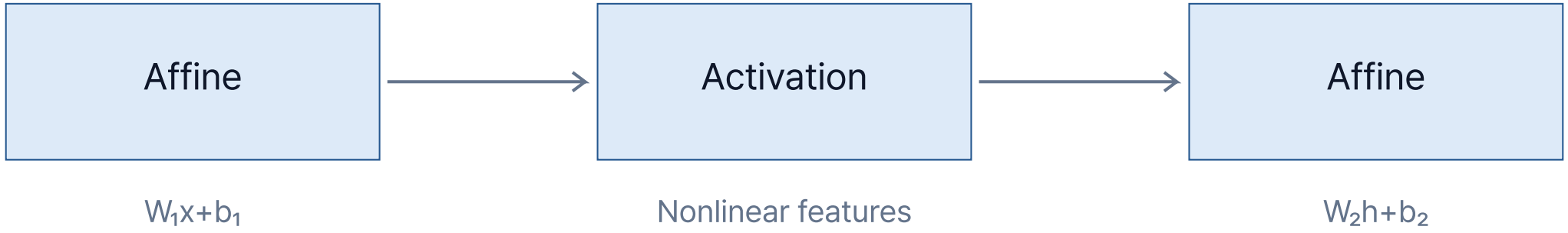
Compute every hidden value before training

```
X=np.array([[0.,0.],[0.,1.],[1.,0.],
[1.,1.]])
h1=np.maximum(0,X.sum(1))
h2=np.maximum(0,X.sum(1)-1)
out=h1-2*h2
print(out)
assert np.array_equal(out,[0,1,1,0])
```

```
[0. 1. 1. 0.]
```

Depth and Composition

Compose simple transformations into a richer function



The second layer combines features built by the first

Width and Parameters

Count trainable values before choosing model size

	Input	Output	Weights + Bias
Layer 1	4	8	40
Layer 2	8	3	27

```
Dense layer parameters = d_in × d_out + d_out
```

Two affine layers here contain sixty-seven parameters

Output Layers

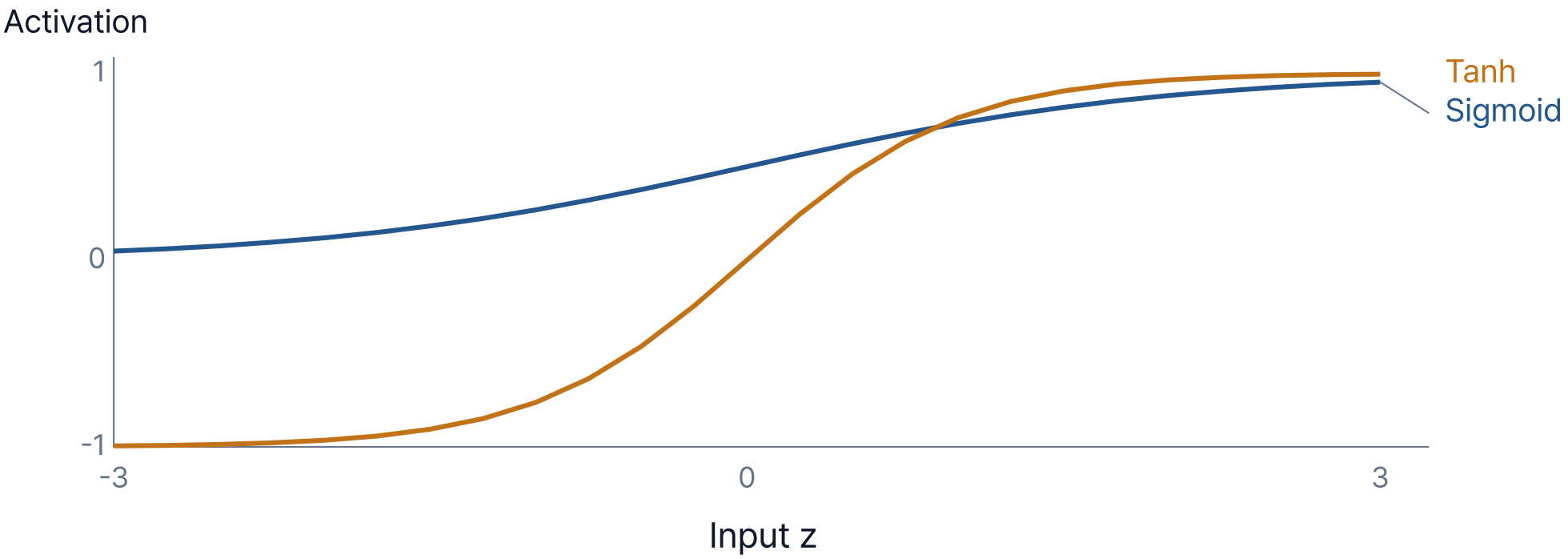
Match the final output to the prediction problem

Task	Output	Loss
Regression	Real values	Squared or absolute loss
Binary	One logit	Binary cross-entropy
Multiclass	C logits	Cross-entropy

The output contract determines the loss interface

Activation Alternatives

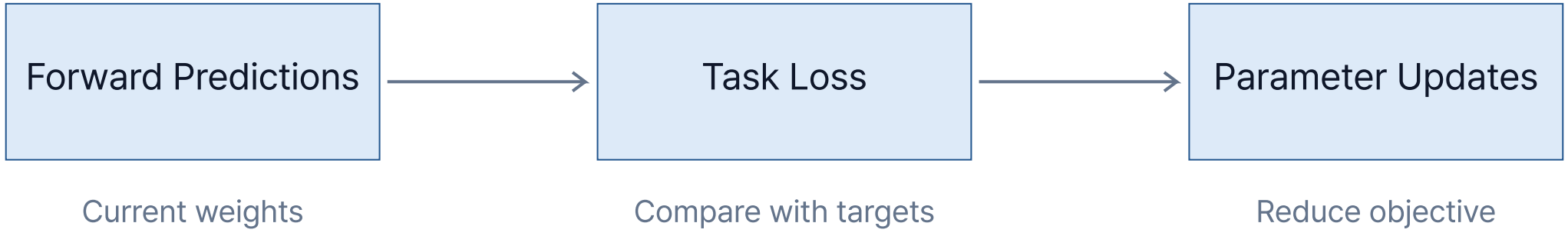
Different nonlinearities change saturation and gradients



Saturating activations flatten for large input magnitudes

Training Objective

Choose weights by minimizing a loss over examples



Representation learning still needs an explicit objective

02 · Differentiable Learning

01 **Learned Features**

Build a nonlinear representation

02 **Differentiable Learning**

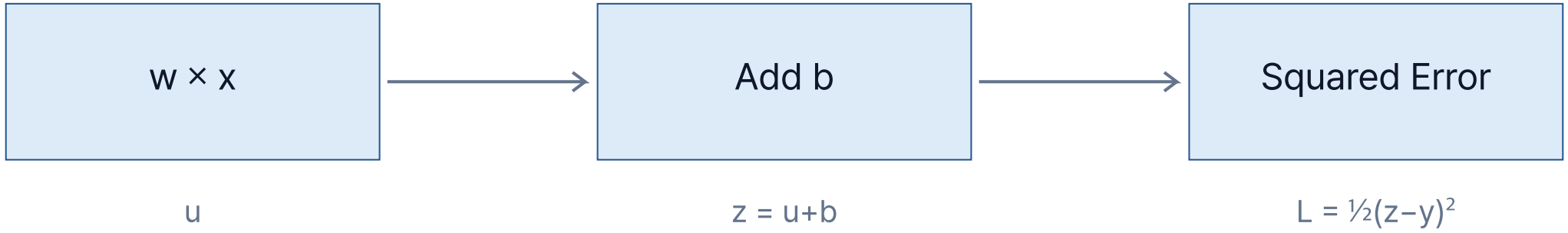
Trace the training computation

03 **Architectural Structure**

Match computation to data

Computation Graphs

Represent intermediate values and dependencies explicitly



The loss depends on w through two intermediate operations

One Forward Pass

Evaluate a scalar model with fixed values

w	x	b	y	z	Loss
1	2	0	3	2	0.5

The current prediction two misses the target three

Local Derivatives

Each operation supplies its own sensitivity

Derivative	Value
L to z	-1
z to w	2
z to b	1

$$\partial L/\partial z = z - y, \ \partial z/\partial w = x$$

The loss falls when this prediction increases

The Chain Rule

Multiply local derivatives along a dependency path

Gradient	Value
∂L/∂w	-2
∂L/∂b	-1

$$\partial L/\partial w = (\partial L/\partial z)(\partial z/\partial w)$$

The weight gradient is minus two for this input

Gradient Accumulation

Shared parameters receive contributions from every use

Path	Gradient Contribution
Example 1	-2
Example 2	1
Sum	-1

Multiple paths add their contributions at a shared parameter

Reverse-Mode Differentiation

Propagate one scalar loss backward through the graph



Start with one

Apply chain rule

Accumulate contributions

One reverse pass computes gradients for many parameters

A Gradient Update

Move parameters against the loss gradient

	Old	Gradient	Step	New
Weight	1	-2	0.1	1.2
Bias	0	-1	0.1	0.1

$$\theta_{new} = \theta - \eta \nabla \theta L$$

The updated parameters raise the prediction toward the target

Finite-Difference Check

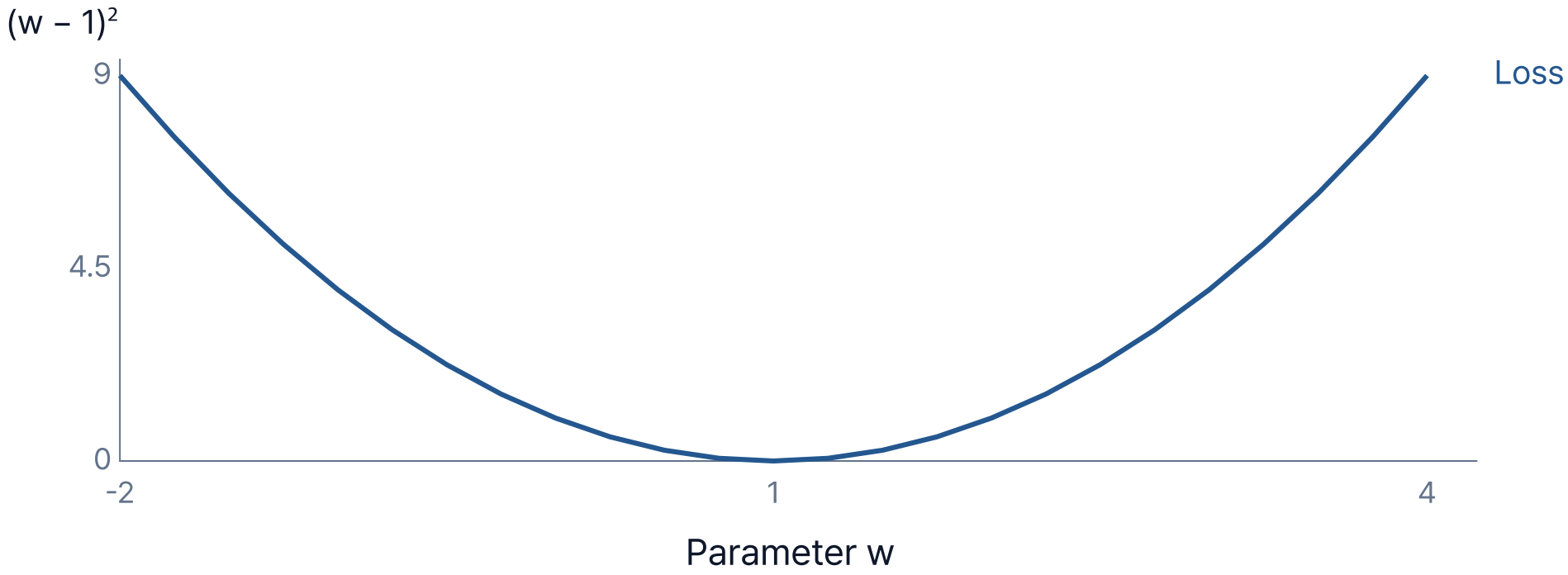
Compare a manual gradient with a numerical perturbation

```
def loss(w): return .5*(w*2-3)**2
w=1.; eps=1e-5
numeric=(loss(w+eps)-loss(w-eps))/(2*eps)
analytic=(w*2-3)*2
print(round(numeric,6), analytic)
assert np.isclose(numeric,analytic)
```

```
-2.0 -2.0
```

Caveat: Learning Rate

A correct gradient can still produce a bad update

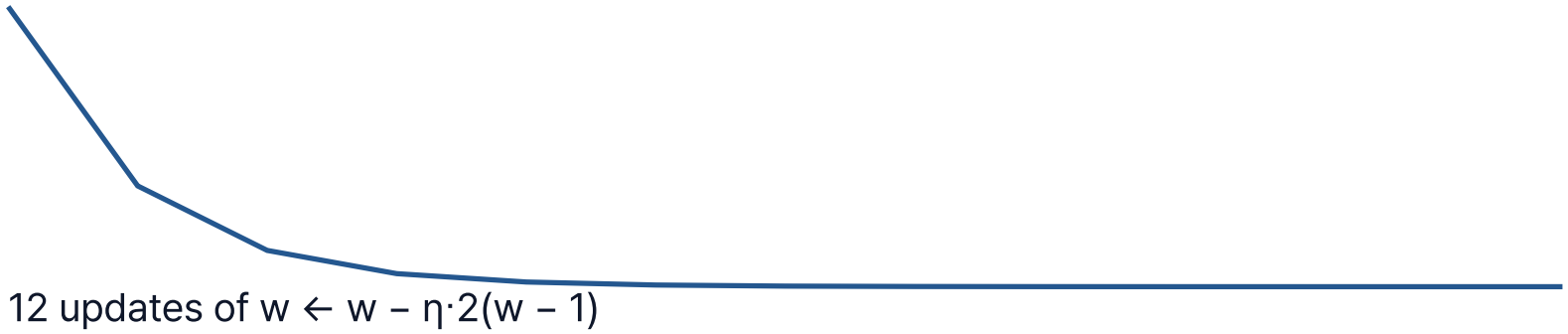


A step can cross the minimum and overshoot

Demo: Learning Rate

Start at w = 4 and compare small, stable, and divergent updates

Learning rate 0.2



12 updates of $w \leftarrow w - \eta \cdot 2(w - 1)$

Learning rate 0.20 | w after 12 steps 1.00653 | loss 0.00004265

For this quadratic, $0 < \eta < 1$ converges; $\eta \geq 1$ does not converge from w = 4

Answer: Stable Update Range

The error contracts only when the multiplier has magnitude below one

$$w_{t+1} - 1 = (1 - 2\eta)(w_t - 1)$$

η	Error Multiplier
0.2	0.6
1.0	-1.0
1.2	-1.4

Stable convergence requires zero less than eta less than one

03 · Architectural Structure

01 Learned Features

Build a nonlinear representation

02 Differentiable Learning

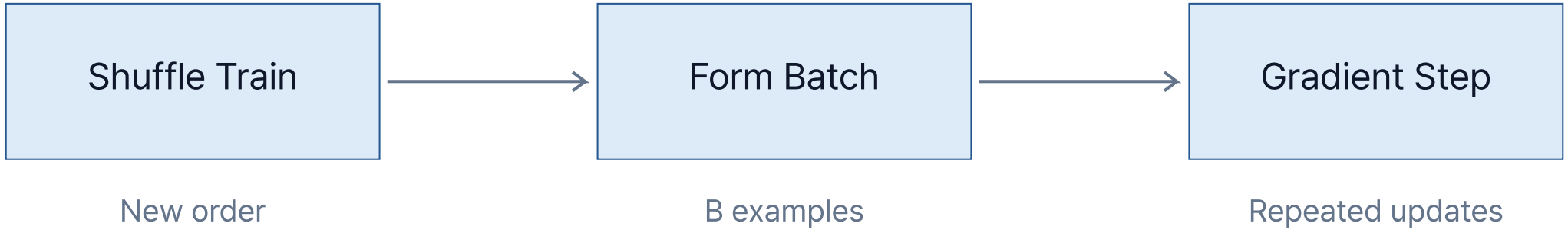
Trace the training computation

03 **Architectural Structure**

Match computation to data

Mini-Batch Learning

Estimate a loss gradient from a subset of examples



Each step uses only the selected batch

Gradient Noise

Small batches produce different update estimates

	g_1	g_2	Mean g
Batch A	-2	1	-0.5
Batch B	-1.8	-0.2	-1.0

Different batches can point in different local directions

Vanishing Gradients

Repeated small derivatives shrink a backward signal

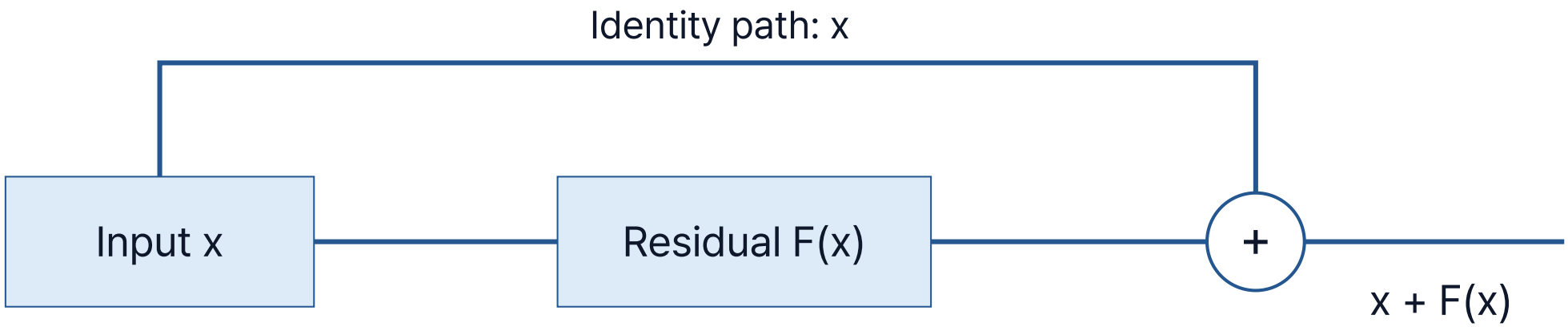


Product of 0.25 derivatives

The product shrinks rapidly across repeated derivatives

Residual Connections

An identity path gives another route for the signal

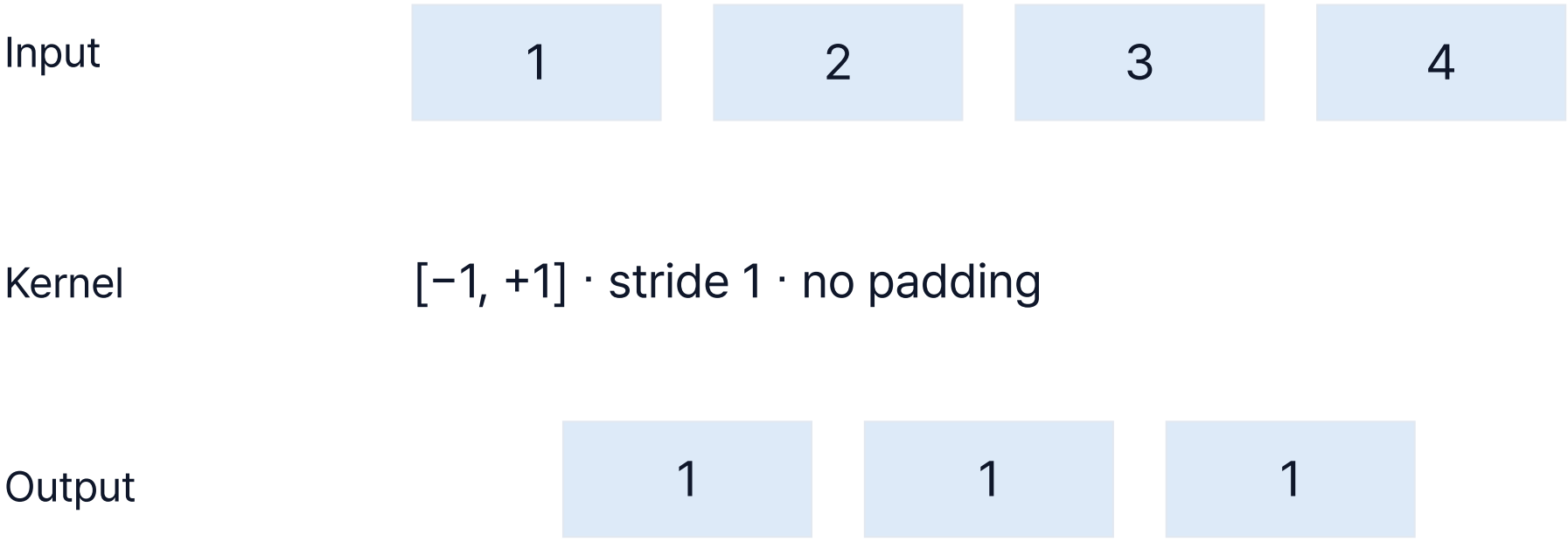


$$dy/dx = 1 + dF(x)/dx$$

The derivative includes an identity contribution

Convolutional Structure

Share a local pattern detector across positions



Valid correlation returns three local differences

Attention as an Architectural Choice

Combine representations using content-dependent weights

	Value A	Value B	Value C
Query A	0.7	0.2	0.1
Query B	0.1	0.6	0.3
Query C	0.2	0.2	0.6

Different queries form different weighted combinations

Architecture and Inductive Bias

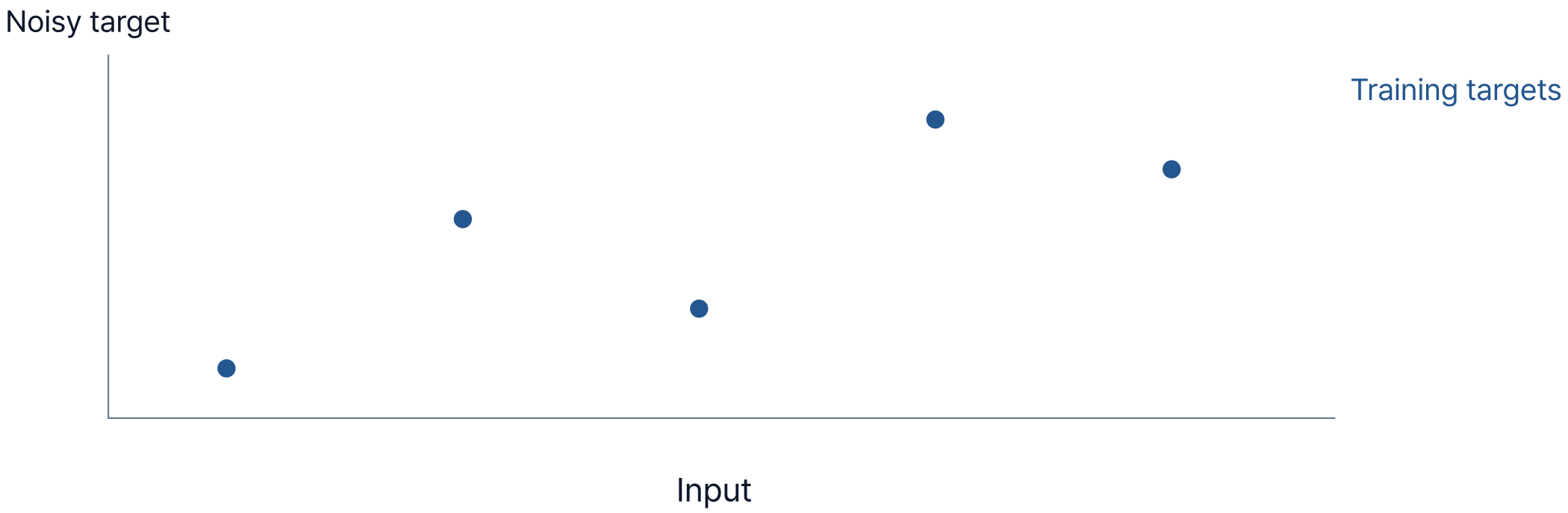
Structure restricts how information is combined

Architecture	Shared Assumption
MLP	Dense feature combinations
CNN	Local patterns repeated in space
Transformer	Content-dependent interactions

An architecture encodes a useful structural preference

Caveat: Capacity and Evidence

More parameters can fit noise as well as signal



A finite noisy sample does not reveal one unique function

Scaling Resources

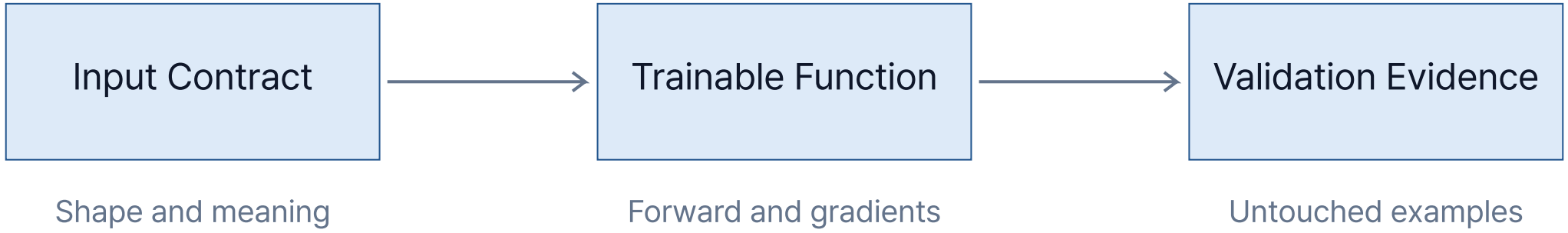
Data, parameters, and compute constrain one training run

Budget	Constraint
Data	Examples and coverage
Parameters	Representational capacity
Compute	Steps and precision

Increasing only one resource can leave another bottleneck

A Complete Learning Contract

Connect the input shape, objective, and validation target



Deep learning remains an explicit, testable computation

Takeaway: Deep Learning

01 Hidden transformations learn task-relevant features

02 Backpropagation applies the chain rule

03 Architecture and validation constrain useful learning